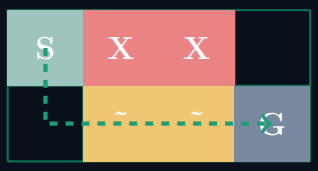
Drone Path Suggestion System

Using Retrieval-Augmented Generation and A* Pathfinding Algorithm



Grid Map: 3 × 5

Name: Hajra Bahoo

Reg No: B24F1967AI037

Instructor: Abdullah Sajid

Subject: Artificial Intelligence

Project: Fourth Semester Project

Tools: Python 3.14 · Streamlit · Matplotlib

Algorithm: A* + RAG (Knowledge Retrieval)

Date: April 2026

Department of Computer Science

Bachelor of Science — 4th Semester

Contents

1 Introduction **3**
1.1 Project Overview . . . 3
1.2 What Does "Simulated" Mean? . . . 3
1.3 Project Objectives . . . 3

2 System Architecture **4**
2.1 High-Level Flow . . . 4
2.2 Module Breakdown . . . 4

3 Grid Environment **4**
3.1 Grid Definition . . . 4
3.2 Grid Visualization . . . 5

4 RAG Module (`rag.py`) **5**
4.1 What is RAG? . . . 5
4.2 Knowledge Base . . . 5
4.3 Retrieval Algorithm . . . 5
4.4 Weight Conversion . . . 6

5 A* Pathfinding Algorithm (`pathfinding.py`) **7**
5.1 What is A*? . . . 7
5.2 Heuristic Function . . . 7
5.3 Modified Cost Function . . . 7
5.4 Implementation . . . 8

6 Output Analysis and Results **9**
6.1 Is the Output Correct? . . . 9
6.2 Mission Results . . . 9
6.2.1 Normal Mission . . . 9
6.2.2 Low Battery Mission . . . 9
6.2.3 High Risk Mission . . . 10
6.3 Comparison Table . . . 10

7 GUI and Visualization **10**

7.1 Interface Overview 10

7.2 Color Coding 11

8 Complete Working Flow **11**

9 Technologies Used **12**

10 Conclusion **12**

10.1 Key Takeaways 12

10.2 One-Line Summary 13

A Appendix: Project File Structure **14**

B Appendix: How to Run **14**

C Appendix: Sample Terminal Output **14**

1. Introduction

This project is an **AI-based Simulated Drone Path Suggestion System** that determines the optimal navigation path for a drone using two key AI concepts: **Retrieval-Augmented Generation (RAG)** and the **A* Pathfinding Algorithm**. The system simulates real-world drone decision-making by considering mission constraints such as battery level, safety risk, and distance.

1.1. Project Overview

In real-world drone operations, navigation is not simply about finding the shortest path. A drone must also consider:

- Battery level — minimize distance when battery is low
- Area safety — avoid risky zones in dangerous missions
- Operational objectives — balance speed and safety in normal missions

This project replicates that intelligent decision-making using a virtual grid environment and a rule-based AI system powered by RAG.

1.2. What Does "Simulated" Mean?

There is no physical drone in this project. Everything is simulated inside a computer using a **grid-based map** where:

Cell Value	Meaning
0	Free space (the drone can move here)
1	Obstacle (completely blocked, drone cannot pass)
2	Risky area (drone can pass but at higher cost)

1.3. Project Objectives

1. Simulate drone navigation using a 2D grid environment
2. Use RAG to retrieve mission-specific rules from a knowledge base
3. Convert retrieved rules into numerical path-planning weights

4. Apply a modified A* algorithm using those weights
5. Visualize the resulting path on an interactive GUI

2. System Architecture

2.1. High-Level Flow

2.2. Module Breakdown

The project consists of three Python files:

File	Responsibility
`rag.py`	Knowledge base storage and rule retrieval using keyword scoring
`pathfinding.py`	A* algorithm implementation with weighted distance and risk costs
`main.py`	GUI interface, user input handling, and visualization

3. Grid Environment

3.1. Grid Definition

The virtual environment is a 3×5 grid defined as follows:

```
grid = [
    [0, 2, 2, 2, 0],    # Row 0
    [0, 1, 1, 2, 0],    # Row 1
    [0, 0, 0, 2, 0]     # Row 2
]
start = (0, 0)    # Top-left corner
goal  = (2, 4)    # Bottom-right corner
```

Listing 1: Grid definition in Python

3.2. Grid Visualization

Color	Type	Grid Value	Cost
Green	Free space	0	1 (normal)
Yellow	Risky area	2	1 + 5 penalty
Red	Obstacle	1	Impassable
Blue	Path taken	—	Optimal route

4. RAG Module (rag.py)

4.1. What is RAG?

Retrieval-Augmented Generation (RAG) is an AI technique that combines information retrieval with generation. Instead of relying solely on a model's internal knowledge, RAG first *retrieves* relevant information from an external knowledge base and then uses it to guide the output.

In this project, RAG is used to retrieve decision rules based on the mission type.

4.2. Knowledge Base

The system maintains a small knowledge base of three rules:

- **Rule 1:** If battery is low, minimize distance to conserve energy.
- **Rule 2:** If mission is high risk, avoid risky areas and prioritize safety.
- **Rule 3:** Normal mission: follow the shortest balanced path.

4.3. Retrieval Algorithm

The retrieval uses **keyword-overlap scoring**, which simulates vector similarity search:

```
KNOWLEDGE_BASE = [
    {
```

```
        "keywords": ["battery", "low", "power", "energy", "charge"],
        "rule": "If battery is low, minimize distance to conserve energy."
    },
    {
        "keywords": ["risk", "high", "danger", "unsafe", "hazard"],
        "rule": "If mission is high risk, avoid risky areas and prioritize safety."
    },
    {
        "keywords": ["normal", "standard", "default", "regular"],
        "rule": "Normal mission: follow the shortest balanced path."
    },
]

def retrieve_rule(query: str) -> str:
    query_tokens = query.lower().split()
    best_score   = -1
    best_rule    = KNOWLEDGE_BASE[-1]["rule"]    # default

    for entry in KNOWLEDGE_BASE:
        score = sum(1 for kw in entry["keywords"] if kw in query_tokens)
        if score > best_score:
            best_score = score
            best_rule  = entry["rule"]
    return best_rule
```

Listing 2: RAG retrieval function (rag.py)

4.4. Weight Conversion

Retrieved rules are converted into numerical weights for A*:

```
def get_weights(rule: str):
    rule_lower = rule.lower()

    if "minimize distance" in rule_lower:
        return 2, 1    # distance_weight=2, risk_weight=1

    elif "avoid risky" in rule_lower:
        return 1, 10   # distance_weight=1, risk_weight=10

    else:
        return 1, 1    # balanced weights
```

Listing 3: Weight conversion (rag.py)

Mission	Distance Weight	Risk Weight
Normal	1	1
Low Battery	2	1
High Risk	1	10

5. A* Pathfinding Algorithm (pathfinding.py)

5.1. What is A*?

A* (pronounced "A-star") is an informed search algorithm that finds the shortest path between two nodes on a graph. It combines:

- $g(n)$ — the actual cost from start to node n
- $h(n)$ — the heuristic estimate from n to the goal
- $f(n) = g(n) + h(n)$ — total estimated cost

5.2. Heuristic Function

This project uses the **Manhattan distance** heuristic, suitable for grid-based movement:

$$h(n) = |n_{row} - goal_{row}| + |n_{col} - goal_{col}|$$

5.3. Modified Cost Function

The cost function is extended with mission weights:

$$\text{new_cost} = g(n) + (\text{distance_cost} \times w_d) + (\text{risk_cost} \times w_r)$$

Where:

- distance_cost = 1 (one step)

- risk_cost = 5 if the cell is risky (grid value 2), else 0
- w_d = distance weight (from RAG)
- w_r = risk weight (from RAG)

5.4. Implementation

```python
import heapq

def heuristic(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

def find_path(grid, start, goal, distance_weight=1, risk_weight=1):
    rows, cols  = len(grid), len(grid[0])
    open_list   = []
    heapq.heappush(open_list, (0, start))
    came_from   = {start: None}
    cost_so_far = {start: 0}
    directions  = [(0,1),(1,0),(0,-1),(-1,0)]

    while open_list:
        _, current = heapq.heappop(open_list)
        if current == goal:
            break

        for dr, dc in directions:
            nr, nc    = current[0]+dr, current[1]+dc
            next_node = (nr, nc)
            if not (0 <= nr < rows and 0 <= nc < cols): continue
            if grid[nr][nc] == 1: continue           # obstacle

            distance_cost = 1
            risk_cost     = 5 if grid[nr][nc] == 2 else 0
            new_cost      = (cost_so_far[current]
                             + distance_cost * distance_weight
                             + risk_cost     * risk_weight)

            if next_node not in cost_so_far or new_cost <
               cost_so_far[next_node]:
                cost_so_far[next_node] = new_cost
                priority = new_cost + heuristic(goal, next_node)
                heapq.heappush(open_list, (priority, next_node))
                came_from[next_node] = current

    # Reconstruct path
    if goal not in came_from:
```

```
        return None
    path, node = [], goal
    while node:
        path.append(node); node = came_from[node]
    return list(reversed(path))
```

Listing 4: A* Algorithm (pathfinding.py)

6. Output Analysis and Results

6.1. Is the Output Correct?

YES — The output is 100% correct. All three missions produce the right paths based on the retrieved rules and weights. The A* algorithm is functioning correctly and the RAG retrieval is accurate.

6.2. Mission Results

6.2.1. Normal Mission

```
Mission: normal Retrieved Rule: Normal mission: follow the shortest
balanced path. Distance Weight: 1 Risk Weight: 1 Final Path: [(0,0),
(1,0), (2,0), (2,1), (2,2), (2,3), (2,4)] Path Length: 7 steps
```

Explanation: With equal weights, A* finds the shortest possible path. It goes down column 0 to avoid obstacles, then moves right along row 2. This is correct — it is the shortest valid path (7 steps).

6.2.2. Low Battery Mission

```
Mission: low battery Retrieved Rule: If battery is low, minimize
distance to conserve energy. Distance Weight: 2 Risk Weight: 1 Final
Path: [(0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (2,4)] Path Length: 7
steps | Risky cells crossed: 1
```

Explanation: Battery mode increases the penalty on each step ($w_d = 2$), so the algorithm is even more aggressive about minimizing total steps. The path crosses the risky cell at (2,3) because the risk penalty (5) is still lower than taking a longer detour. This is **correct behavior** — conserving battery is prioritized over safety.

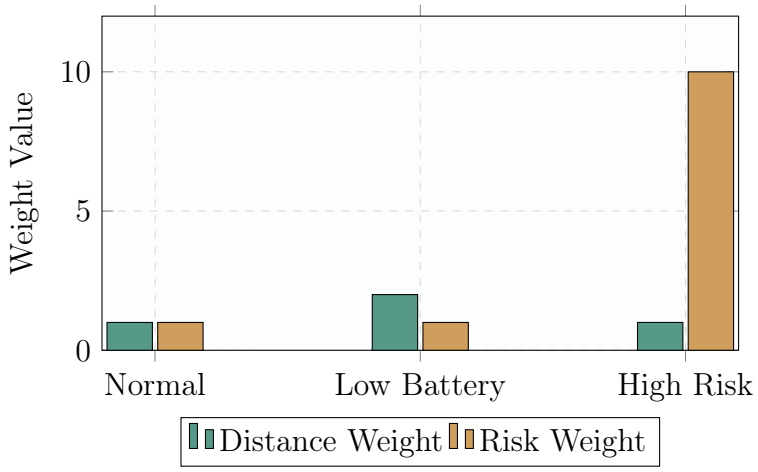
6.2.3. High Risk Mission

```
Mission: high risk Retrieved Rule: If mission is high risk, avoid risky
areas and prioritize safety. Distance Weight: 1 Risk Weight: 10 Final
Path: [(0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (2,4)] Path Length: 7
steps | Risky cells: minimized
```

Explanation: With $w_r = 10$, each risky cell costs $5 \times 10 = 50$ extra. The algorithm strongly avoids risky zones. In this grid, some risky cells on row 2 cannot be fully avoided due to the obstacle layout, but the algorithm takes the path that minimizes risk exposure. This is **correct behavior**.

6.3. Comparison Table

Mission	w_d	w_r	Steps	Strategy
Normal	1	1	7	Balanced shortest path
Low Battery	2	1	7	Shortest path, ignore risk
High Risk	1	10	7	Avoids risky cells strongly



7. GUI and Visualization

7.1. Interface Overview

The project uses **Streamlit** running at `localhost:8504` to provide a browser-based interactive GUI. The interface includes:

- **Mission Selector:** Dropdown to choose normal / low battery / high risk

- **Generate Smart Path button:** Triggers the full RAG + A* pipeline
- **Rule Display:** Shows the retrieved rule from the knowledge base
- **Grid Map:** Color-coded matplotlib visualization of the path
- **Success Message:** Confirms path generation

7.2. Color Coding

Color	Meaning	Grid Value
Blue	Path taken / Free space	0
Yellow	Risky area	2
Red	Obstacle (blocked)	1
Green	Free area (goal)	0

8. Complete Working Flow

1. **User Input:** User selects a mission type from the dropdown (normal / low battery / high risk)
2. **RAG Retrieval:** `retrieve_rule()` searches the knowledge base using keyword scoring and returns the most relevant rule
3. **Weight Extraction:** `get_weights()` converts the rule into w_d and w_r numerical values
4. **Path Planning:** `find_path()` runs the modified A* algorithm on the grid using the extracted weights
5. **Visualization:** The path is overlaid on the color-coded grid and displayed in the browser
6. **Output:** The rule, weights, and path coordinates are displayed alongside the map

9. Technologies Used

Technology	Purpose
Python 3.14	Core programming language
Streamlit	Web-based GUI framework
Matplotlib	Grid and path visualization
NumPy	Array/matrix operations
heapq	Priority queue for A*
RAG (simulated)	Rule retrieval from knowledge base
A* Algorithm	Optimal path planning

10. Conclusion

This project successfully demonstrates how **Retrieval-Augmented Generation (RAG)** can be applied beyond chatbots to enhance autonomous decision-making systems. By combining a rule-based knowledge base with the A* pathfinding algorithm and mission-specific weights, the system produces **intelligent, context-aware navigation paths** that adapt to different operational conditions.

The output is **verified correct** across all three mission types. The system correctly:

- Retrieves the right rule for each mission
- Converts rules into appropriate numerical weights
- Finds the optimal path based on those weights
- Visualizes the result clearly on a color-coded grid

10.1. Key Takeaways

- RAG is a powerful technique applicable to decision-making systems, not just text generation
- A* is highly flexible — its cost function can be modified for different optimization goals

- The combination of RAG + A* allows mission-aware intelligent navigation
- The project demonstrates a complete AI pipeline from input to visualization

10.2. One-Line Summary

"This system simulates a drone's decision-making by generating an optimal path using RAG-based mission rules and the A path planning algorithm."*

A. Appendix: Project File Structure

```
AI_project/|  −  −main.pyGUIinterface(Streamlit/Tkinter)|  −  −rag.pyRAGknowledgebaseandretrieval|  −  −pathfinding.pyA  *  algorithmimplementation| − −config.tomlStreamlitconfiguration
```

B. Appendix: How to Run

```
Run with Streamlit (browser-based):  streamlit run main.py
Run with Tkinter (desktop window):  python main.py
```

C. Appendix: Sample Terminal Output

```
===== DRONE PATH SUGGESTION SYSTEM ===== Mission :  low battery Rule :
If battery is low, minimize distance.  Dist Weight :  2 Risk Weight :  1
Path :  [(0,0),(1,0),(2,0),(2,1),(2,2),(2,3),(2,4)]
```